ShadowDesk

Internal Helpdesk & Support Ticket Management System

Project Title:	ShadowDesk – Internal Helpdesk & Support Ticket Management System	**Internship Organization:**	Shadowfox
Student Name:	Anurag Acharya	**Mentor Name:**	[Mentor Name]
College / Institution:	JECRC University	**Department:**	Computer Science & Engineering (CSE)
Internship Duration:	01/10/2026 – 31/10/2026	**Submission Date:**	30/10/2026

1. Abstract

ShadowDesk is a robust, modular, and dependency-free internal support ticket management system developed in Core Java (Java 8+ compatible). In enterprise logistics and technology operations, internal support teams frequently face unstructured issue reporting, lack of priority visibility, and lost audit trails. ShadowDesk addresses this challenge by providing a centralized application that allows internal personnel to log support tickets with unique formatted IDs (SF-1001, SF-1002), track issues through an enforced lifecycle state machine (OPEN → IN_PROGRESS → RESOLVED → CLOSED), mandate resolution notes, dynamically compute operational metrics, and persist records using crash-resilient RFC-4180 CSV storage. Engineered strictly adhering to Object-Oriented Programming (OOP) principles and layered architectural separation, the application delivers enterprise-grade reliability without external third-party dependencies.

2. Problem Statement

Modern corporate and logistics environments handle hundreds of daily operational requests—such as device connectivity failures, warehouse scanner breakdowns, driver app authentication blockers, and account permissions. When organizations manage these requests informally across chat channels, emails, or spreadsheets, several critical bottlenecks occur:

- **Lack of Structured Intake:** Unformatted requests lack mandatory operational attributes (category, urgency, requester contact), causing prolonged diagnostic delays.
- **Absence of Priority Differentiation:** Critical operational blockers cannot be separated from routine inquiries without systematic categorization.
- **Uncontrolled State Transitions:** Informal tracking allows tickets to be prematurely marked resolved or closed without documented root causes or fixes.
- **Data Loss & Sequence Corruption:** Non-persistent or naive text storage frequently suffers from data corruption when user input contains commas, quotes, or line breaks.

3. Project Objectives

The primary objective of the ShadowDesk project is to build an extensible, production-quality Core Java helpdesk solution that guarantees operational integrity:

1. **Automated Identification:** Implement thread-safe sequential ticket ID generation (e.g., SF-1001) with sequence recovery across application restarts.
2. **Enforced State Workflow:** Enforce valid state transitions at the business layer to reject invalid status jumps (e.g., closing an uncompleted ticket).
3. **Comprehensive Audit Logging:** Enforce mandatory resolution notes upon resolution and preserve historical notes when tickets are reopened.
4. **Dynamic Operational Analytics:** Provide live metric aggregations (total, open, in-progress, resolved, closed, critical) computed directly from memory.
5. **Safe File Persistence:** Implement RFC-4180 compliant CSV storage with atomic temporary file swapping to prevent data corruption during unexpected failures.

4. Technology Stack & Environment

ShadowDesk is implemented strictly in **Core Java SE** without external third-party frameworks (such as Spring, Hibernate, or external JSON libraries), guaranteeing universal execution on any Java 8+ runtime environment. Key platform components include: **Java SE 8+** compatibility, **Java Collections Framework** (LinkedHashMap for O(1) key lookups with preserved insertion order, ArrayList for defensive copying and filtering), **Java 8 Date/Time API** (java.time.LocalDateTime and DateTimeFormatter for immutable ISO persistence and human-readable formatting), **Atomic Concurrency** (java.util.concurrent.atomic.AtomicInteger for thread-safe sequential ID generation), and **Java NIO File Channels** (java.nio.file.Files with atomic move operations for crash-safe persistence).

5. System Design & Architecture

The project follows a disciplined multi-tiered architectural model ensuring single responsibility and high cohesion:

Architectural Layer	Component / Class	Core Responsibilities & Implementation Details
Presentation Layer	Main.java InputUtil.java	Coordinates the 12-item interactive console menu; captures and validates user input; formats data tables; isolates console I/O from domain business logic.
Service Layer	TicketService.java	Authoritative business engine. Enforces state transitions, manages in-memory LinkedHashMap, validates creation/update constraints, and computes dynamic metrics.
Persistence Layer	TicketRepository.java	Reads and writes RFC-4180 compliant CSV (data/tickets.csv). Handles quotes, commas, and newlines; writes to temporary files before atomic file swapping.
Domain Model	Ticket.java Enums	Encapsulates ticket attributes, audit timestamps (createdAt, updatedAt), and domain methods. Enums: TicketCategory, TicketPriority, TicketStatus.

6. Major Implemented Features

- **Sequential ID Management:** Automatic generation of readable identifiers (SF-1001, SF-1002). Recovers highest counter from persistent storage on startup to eliminate ID reuse.
- **Categorization & Prioritization:** Supports five categories (*Technical, Delivery Operations, Account Access, Hardware, Other*) and four priorities (*Low, Medium, High, Critical*).
- **Multi-Attribute Filtering & Search:** Case-insensitive O(1) ID search plus multi-criteria filtering by status, priority, category, or creator name.
- **Dynamic Metric Dashboard:** Real-time dynamic aggregation of total, open, in-progress, resolved, closed, and high/critical tickets directly from active memory.
- **Defensive Console Controls:** Line-based scanning avoids Scanner integer buffer bugs; handles non-numeric characters and out-of-range choices gracefully.

7. Ticket Lifecycle Workflow

Ticket lifecycles follow a strict finite-state machine governed by TicketService.getAvailableTransitions():

Current State	Permitted Transitions	Business Constraints & Validation Rules
OPEN	→ IN_PROGRESS → RESOLVED	Initial state upon ticket creation. Can transition to IN_PROGRESS when assigned, or directly to RESOLVED if fixed immediately with resolution notes.
IN_PROGRESS	→ RESOLVED	Work actively underway. Can only move to RESOLVED; cannot jump directly to CLOSED.
RESOLVED	→ CLOSED → OPEN (Reopen)	Mandatory non-blank resolution notes required. Can be officially CLOSED by admin/requester or moved back to OPEN if the issue persists.
CLOSED	→ OPEN (Reopen)	Terminal state. Direct moves to IN_PROGRESS or RESOLVED are prohibited. Reopening transitions to OPEN and archives previous resolution notes.

8. Object-Oriented Programming Principles Applied

- **Encapsulation:** All 11 attributes in Ticket.java are private. Modifications occur strictly through domain mutators (touch(), updateDetails()).
- **Custom Domain Exceptions:** Checked exceptions (TicketNotFoundException, InvalidTicketOperationException) clearly separate business errors from technical bugs.
- **Decoupled Persistence:** Storage logic is wholly isolated in TicketRepository, allowing future migration to SQL databases without altering service or presentation code.

9. Validation and Testing

The application was tested through an automated, dependency-free test runner (TicketSystemTestSuite.java) executing 17 comprehensive functional, validation, and persistence test cases. All 17 tests passed with zero failures:

#	Test Case Description	Expected Validation / Business Behavior	Result
1	Valid ticket creation	Generates SF-1001 with default OPEN status and valid timestamps	**PASS**
2	Blank title rejection	Throws IllegalArgumentException on empty or whitespace title	**PASS**
3	Invalid priority/category handling	Rejects null or unlisted categories and priorities safely	**PASS**
4	Unique sequential IDs	Generates SF-1001, SF-1002, SF-1003 with zero collisions	**PASS**
5	Search existing ticket	O(1) retrieval with full detail matching and case-insensitivity	**PASS**
6	Search nonexistent ticket	Catches TicketNotFoundException gracefully without application crash	**PASS**
7	Atomic ticket update	Updates fields and refreshes updatedAt timestamp atomically	**PASS**
8	Resolution notes enforcement	Blocks resolution with blank notes; records resolution text on success	**PASS**
9	Reject closing OPEN ticket	Throws InvalidTicketOperationException; requires RESOLVED state	**PASS**
10	Full lifecycle transition	Verifies OPEN → IN_PROGRESS → RESOLVED → CLOSED flow	**PASS**
11	Reject invalid transition	Blocks direct move from CLOSED to IN_PROGRESS without reopening	**PASS**
12	Reopening CLOSED ticket	Reverts status to OPEN while archiving prior resolution notes	**PASS**
13	Multi-attribute filter	Correctly filters by Category, Priority, Status, and Creator	**PASS**
14	Dynamic dashboard stats	Accurately aggregates live ticket counts across status and priorities	**PASS**
15	Persistence across restarts	Saves to CSV, reloads tickets, and recovers sequential ID counter	**PASS**
16	Console input resilience	Handles alphabetic and out-of-range menu inputs without termination	**PASS**
17	Storage load failure safety	Refuses to overwrite valid CSV storage with empty collection on error	**PASS**

10. Project Outcome

The project delivered a complete, dependable Core Java internal helpdesk system featuring three distinct interfaces: an **Interactive Console CLI** (run.bat), a **Modern Web UI Dashboard** (run-web.bat powered by built-in HttpServer on http://localhost:8080), and a **Desktop Swing GUI** (run-gui.bat). The application meets all functional requirements with zero external runtime dependencies.

11. Limitations and Future Scope

• **Current Limitations:** Data storage relies on local CSV flat files, limiting concurrent multi-machine operations. Authentication operates via predefined employee/agent metadata without password security.
• **Future Scope:** Transition to an enterprise relational database (PostgreSQL/MySQL) with connection pooling; implement JWT/RBAC role authentication; integrate automated email/SMS webhook notifications on status changes; and introduce SLA countdown timers with automated escalation.

12. Conclusion

The ShadowDesk project successfully fulfills all internship objectives set forth at Shadowfox. It showcases the practical application of Core Java, strong encapsulation, layered decoupling, state-machine validation, and crash-resilient file persistence in solving real-world enterprise operational bottlenecks.